

Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»
Кафедра ИИТ

Лабораторная работа №2

За седьмой семестр

По дисциплине: «Модели решения задач в интеллектуальных системах»

Тема: «Многослойный персептрон для задачи «исключающее ИЛИ» с числовыми классами: сравнение функций потерь MSE и Binary Cross-Entropy»

Выполнил:

Студент 4 курса
Группы ИИ-26(1)
Прокопюк А.Д.

Проверил:

Никитин С.К.

Брест 2026

Цель работы: На основе многослойного персептрона архитектуры 2-2-1, реализованного в Лабораторной работе №1 для задачи XOR с числовыми классами c_0 и c_1 из диапазона $[-10; 10]$, реализовать альтернативную функцию потерь – бинарную кросс-энтропию (Binary Cross-Entropy, BCE) – и провести сравнительный анализ обучения сети с MSE (Лабораторная работа №1) и с BCE по скорости сходимости, качеству обучения и устойчивости к выбору начальных весов.

Вариант 12 ($c_0 = -10$, $c_1 = 9$)

A	B	$A \oplus B$
-10	-10	-10
-10	9	9
9	-10	9
9	9	-10

Задачи лабораторной работы

- Взять за основу код многослойного персептрона 2-2-1 из Лабораторной работы №1.
 - Поскольку BCE определена для выходов в диапазоне $(0; 1)$, а целевые значения в вашем варианте – произвольные числа c_0 и c_1 из диапазона $[-10; 10]$, реализовать нормализацию (масштабирование) целевых значений перед обучением и обратное преобразование выхода сети после обучения:
 - привести c_0 и c_1 к бинарным меткам 0 и 1;
 - обучать сеть на нормализованных целевых значениях с функцией потерь BCE;
 - при выводе результата преобразовывать выход сети $\hat{y} \in (0; 1)$ обратно в шкалу $[c_0; c_1]$.
 - Реализовать корректный градиент для связки BCE + сигмоида на выходном слое; для скрытого слоя градиент передаётся по стандартному правилу цепочки.
 - Обучить сеть на полной таблице истинности своего варианта (4 примера) в двух конфигурациях:
 - Конфигурация А (базовая, из ЛР №1):** MLP 2-2-1, функция потерь MSE, целевые значения – исходные c_0/c_1 (без нормализации), выходной слой – сигмоида. Результаты можно взять из отчёта по ЛР №1 или, при необходимости, повторить обучение с фиксацией всех метрик.
 - Конфигурация Б (новая):** MLP 2-2-1, функция потерь BCE, целевые значения – нормализованные (0/1), с последующим пересчётом выхода в шкалу $[c_0; c_1]$.
- Критерий остановки для обеих конфигураций одинаков – суммарная ошибка $E_s \leq E_e$ (используйте то же значение E_e , что и в ЛР №1, либо согласуйте новое значение отдельно для MSE и для BCE, поскольку это разные по своей природе величины – обоснуйте выбор в отчёте).
- Для каждой конфигурации зафиксировать:
 - количество эпох до достижения критерия остановки (либо факт недостижения за максимально допустимое число эпох);
 - итоговую суммарную ошибку;
 - ассигасу на всех 4 примерах таблицы истинности;
 - среднюю абсолютную ошибку в исходной шкале $[c_0; c_1]$.
 - Провести не менее 5 запусков обучения для каждой конфигурации с разной случайной инициализацией весов (менять seed) и сравнить устойчивость сходимости: в

скольких из 5 запусков сеть достигла критерия останова, и какой разброс по числу эпох наблюдается.

7. Построить общий график зависимости суммарной ошибки от номера эпохи для конфигурации А (MSE) и конфигурации Б (BCE) на одних осях (для представительного запуска каждой конфигурации).

8. Реализовать блок визуализации результатов, включающий:

- **График сходимости** – суммарная ошибка E_s от номера эпохи, конфигурации А и Б на одних осях, разными цветами/стилями линий, с обозначением порога E_e .
- **Диаграмму разброса числа эпох по 5 запускам** (bar chart или scatter) для конфигурации А и конфигурации Б – наглядно показывающую устойчивость сходимости из п. 6 (например, столбики числа эпох для каждого из 5 seed, сгруппированные по конфигурации; запуски, не достигшие критерия останова, отметить отдельно, например штриховкой или отдельным цветом).
- **Визуализацию разделяющей поверхности скрытого слоя** – для представительного обученного экземпляра каждой конфигурации построить на плоскости (А, В) карту выходного значения сети (heatmap или contour plot) с наложенными точками обучающей выборки (4 примера таблицы истинности), закрашенными по истинному классу. Диапазон осей А и В – $[-10; 10]$ (весь допустимый диапазон входных значений). Для конфигурации Б цветовую шкалу можно строить как по нормализованному $\hat{y} \in (0; 1)$, так и по пересчитанному значению в шкале $[c_0; c_1]$ – выбрать более наглядный вариант и обосновать выбор.
- **Диаграмму сравнения точности восстановления шкалы** – столбчатую диаграмму средней абсолютной ошибки для конфигурации А и конфигурации Б, чтобы наглядно сопоставить точность регрессии к исходной шкале $[c_0; c_1]$.

Каждый график должен иметь подписанные оси, заголовок и легенду (где применимо). Разделяющую поверхность и графики точности рекомендуется разместить рядом (например, subplot 2×2 или отдельными рисунками) для удобства сравнения в отчёте.

9. Реализовать режим функционирования сети: пользователь вводит пару чисел (А, В) из диапазона $[-10; 10]$ (не обязательно совпадающих с c_0 и c_1). Сеть должна выводить:

- выход в нормализованной шкале $\hat{y} \in (0; 1)$ (с точностью до 4 знаков);
- выход, пересчитанный в исходную шкалу $[c_0; c_1]$ по формуле из п. 2;
- к какому классу (c_0 или c_1) ближе полученное значение.

Продемонстрировать работу режима функционирования на всех 4 примерах исходной таблицы истинности и не менее чем на 2–3 дополнительных парах (А, В), не входящих в обучающую выборку.

2. Описание изменений в архитектуре и алгоритме обучения по сравнению с ЛР №1: формула нормализации целевых значений, формула обратного пересчёта выхода, вывод градиента для BCE + сигмоида.

Изменений в архитектуре сети (2-2-1) нет, были изменены целевые значения и алгоритм обучения. Для BCE целевые значения нормализуются по формуле:

$$y_{norm} = \frac{y - c_0}{c_1 - c_0}$$

А после работы сети выполняется обратный перерасчет:

$$y = c_0 + y_{norm}(c_1 - c_0)$$

Для выхода с сигмодой и BCE, функция ошибки имеет вид:

$$CE = -(e_j \ln y_j + (1 - e_j) \ln(1 - y_j))$$

После дифференцирования с учетом производной сигмоиды, градиент упрощается до

$$\frac{\partial E}{\partial s_j} = y_j - e_j$$

3. Код программы:

```
import numpy as np
import matplotlib.pyplot as plt

def sigmoid(wsum):
    return 1 / (1 + np.exp(-wsum))

def der_sigmoid(y):
    return y * (1 - y)

def mse(yj, ej):
    return 0.5 * (yj - ej) ** 2

def bce(yj, ej):
    eps = 1e-12
    yj = np.clip(yj, eps, 1 - eps)
    return -(ej * np.log(yj) + (1 - ej) * np.log(1 - yj))

def normalize(val, min_val=-10, max_val=9):
    return (val - min_val) / (max_val - min_val)

def denormalize(val, min_val=-10, max_val=9):
    return min_val + val * (max_val - min_val)

def calculate_accuracy(net, X_norm, y_raw_true, c0=-10, c1=9):
    correct = 0
    threshold_mid = (c0 + c1) / 2

    for i in range(len(X_norm)):
        pred_norm = net.forward(X_norm[i:i+1])[0][0]
        pred_denorm = denormalize(pred_norm, min_val=c0, max_val=c1)

        assigned_class = c1 if pred_denorm >= threshold_mid else c0
        true_class = y_raw_true[i][0]

        if assigned_class == true_class:
            correct += 1

    return (correct / len(X_norm)) * 100

def calculate_mae(net, X_norm, y_raw, c0=-10, c1=9):
    total_error = 0

    for i in range(len(X_norm)):
        pred_norm = net.forward(X_norm[i:i+1])[0][0]
        pred_raw = denormalize(pred_norm, min_val=c0, max_val=c1)
```

```

        total_error += abs(pred_raw - y_raw[i][0])

    return total_error / len(X_norm)

class NN:
    def __init__(self, layer_sizes):
        self.layer_sizes = layer_sizes
        self.num_layers = len(layer_sizes)

        self.weights = []
        self.thresholds = []

        for i in range(self.num_layers - 1):
            limit = 1 / np.sqrt(layer_sizes[i])
            w = np.random.uniform(-limit, limit, (layer_sizes[i], layer_sizes[i +
1]))

            T = np.zeros((1, layer_sizes[i + 1]))

            self.weights.append(w)
            self.thresholds.append(T)

    def forward(self, X):
        self.activations = [X]
        current_x = X    #y_j

        for i in range(self.num_layers-1):
            wsum = np.dot(current_x, self.weights[i]) - self.thresholds[i]
            current_x = sigmoid(wsum)
            self.activations.append(current_x)

        return current_x

    def backward(self, ethalon, lr):
        error = (self.activations[-1] - ethalon) * der_sigmoid(self.activations[-
1])

        for i in range(self.num_layers-2, -1, -1):
            inputs = self.activations[i]

            if i > 0:
                prev_error = (np.dot(error, self.weights[i].T) *
der_sigmoid(self.activations[i]))

            self.weights[i] -= np.dot(inputs.T, error) * lr
            self.thresholds[i] += np.sum(error, axis=0, keepdims=True) * lr

            if i > 0:
                error = prev_error

    def bce_backward(self, ethalon, lr):
        error = self.activations[-1] - ethalon

```

```

        for i in range(self.num_layers - 2, -1, -1):
            inputs = self.activations[i]
            if i > 0:
                prev_error = (np.dot(error, self.weights[i].T) *
der_sigmoid(self.activations[i]))

            self.weights[i] -= np.dot(inputs.T, error) * lr
            self.thresholds[i] += np.sum(error, axis=0, keepdims=True) * lr

            if i > 0:
                error = prev_error

def train_network(layer_sizes, X, y, epochs=20000, lr=0.5, target_error=0.0001):
    net = NN(layer_sizes)
    error_history = []
    converged = False

    final_epoch = epochs
    for epoch in range(epochs):
        total_error = 0
        for i in range(len(X)):
            pred = net.forward(X[i:i+1])
            total_error += np.sum(mse(pred, y[i:i+1]))
            net.backward(y[i:i+1], lr)

        error_history.append(total_error)

        if epoch % 10000 == 0:
            acc = calculate_accuracy(net, X, y_raw, c0, c1)
            mae = calculate_mae(net, X, y_raw, c0, c1)
            print(f"Epoch {epoch}, Error: {total_error:.5f}, Accuracy:
{acc:.1f}%, MAE: {mae}")

            if total_error <= target_error:
                final_epoch = epoch + 1
                converged = True
                break

    mae = calculate_mae(net, X, y_raw, c0, c1)

    return net, error_history, final_epoch, error_history[-1], mae, converged

def bce_train_network(layer_sizes, X, y, epochs=20000, lr=0.5,
target_error=0.0001):
    net = NN(layer_sizes)
    error_history = []
    converged = False

    final_epoch = epochs
    for epoch in range(epochs):

```

```

total_error = 0
for i in range(len(X)):
    pred = net.forward(X[i:i+1])
    total_error += np.sum(bce(pred, y[i:i+1]))
    net.bce_backward(y[i:i+1], lr)

error_history.append(total_error)

if epoch % 10000 == 0:
    acc = calculate_accuracy(net, X, y_raw, c0, c1)
    mae = calculate_mae(net, X, y_raw, c0, c1)
    print(f"Epoch {epoch}, Error: {total_error:.5f}, Accuracy: {acc:.1f}%, MAE: {mae}")

if total_error <= target_error:
    final_epoch = epoch + 1
    converged = True
    break

mae = calculate_mae(net, X, y_raw, c0, c1)

return net, error_history, final_epoch, error_history[-1], mae, converged

def predict_point(net, a, b, c0=-10, c1=9):
    raw_input = np.array([[a, b]])
    norm_input = normalize(raw_input, min_val=c0, max_val=c1)

    pred_norm = net.forward(norm_input)[0][0]

    pred_denorm = denormalize(pred_norm, min_val=c0, max_val=c1)

    threshold_mid = (c0 + c1) / 2
    assigned_class = c1 if pred_denorm >= threshold_mid else c0

    print(
        f"Input: A={a}, B={b}\n"
        f"Normalized output: {pred_norm:.4f}\n"
        f"Original scale output: {pred_denorm:.4f}\n"
        f"Predicted class: {assigned_class}"
    )

def plot_all_histories(history_mlp_mse, history_mlp_bce,
                      history_slp_mse, history_slp_bce):

    histories = [
        (history_mlp_mse, "MLP 2-2-1 + MSE"),
        (history_mlp_bce, "MLP 2-2-1 + BCE"),
        (history_slp_mse, "SLP 2-1 + MSE"),
        (history_slp_bce, "SLP 2-1 + BCE")
    ]

```

```

for history, title in histories:
    plt.figure(figsize=(10, 6))

    plt.plot(
        range(1, len(history) + 1),
        history,
        label="Sum error"
    )

    plt.xlabel("Epochs")
    plt.ylabel("Sum error")
    plt.title(title)
    plt.grid(True)
    plt.legend()
    plt.tight_layout()
    plt.show()

def plot_convergence(history_mse, history_bce, target_error=0.0001):
    plt.figure(figsize=(10, 6))

    plt.plot(
        range(1, len(history_mse) + 1),
        history_mse,
        label="MSE"
    )

    plt.plot(
        range(1, len(history_bce) + 1),
        history_bce,
        label="BCE"
    )

    plt.axhline(
        y=target_error,
        linestyle="--",
        label=f"Target Ee = {target_error}"
    )

    plt.xlabel("Epochs")
    plt.ylabel("Es")
    plt.title("Convergence")
    plt.legend()
    plt.grid(True)
    plt.tight_layout()
    plt.show()

def plot_epoch_scatter(mse_epochs, mse_converged,
                       bce_epochs, bce_converged):

    x = np.arange(5)
    width = 0.35

```

```

plt.figure(figsize=(10, 6))

plt.bar(
    x - width / 2,
    mse_epochs,
    width,
    label="MSE"
)

plt.bar(
    x + width / 2,
    bce_epochs,
    width,
    label="BCE"
)

plt.xlabel("№ launch")
plt.ylabel("Epochs")
plt.title("Scatter")
plt.xticks(x, [f"Seed {seed}" for seed in [1, 2, 3, 4, 5]])
plt.legend()
plt.grid(axis="y", alpha=0.3)

plt.tight_layout()
plt.show()

def plot_decision_boundary(net, title, c0=-10, c1=9):
    A = np.linspace(-10, 10, 200)
    B = np.linspace(-10, 10, 200)

    AA, BB = np.meshgrid(A, B)

    points = np.column_stack((AA.ravel(), BB.ravel()))

    points_norm = normalize(points, min_val=c0, max_val=c1)

    predictions_norm = net.forward(points_norm).reshape(AA.shape)

    predictions = denormalize(
        predictions_norm,
        min_val=c0,
        max_val=c1
    )

    plt.figure(figsize=(8, 7))

    contour = plt.contourf(
        AA,
        BB,
        predictions,

```

```

        levels=50,
        cmap="viridis"
    )

    plt.colorbar(contour, label="Network output")

    plt.scatter(
        X_raw[:, 0],
        X_raw[:, 1],
        c=y_raw[:, 0],
        cmap="coolwarm",
        edgecolors="black",
        s=100
    )

    plt.xlim(-10, 10)
    plt.ylim(-10, 10)

    plt.xlabel("A")
    plt.ylabel("B")
    plt.title(title)

    plt.grid(alpha=0.2)
    plt.tight_layout()
    plt.show()

def plot_mae_comparison(net_mlp_mse, net_mlp_bce,
                        net_slp_mse, net_slp_bce,
                        c0=-10, c1=9):

    names = [
        "MLP + MSE",
        "MLP + BCE",
        "SLP + MSE",
        "SLP + BCE"
    ]

    values = [
        calculate_mae(net_mlp_mse, X, y_raw, c0, c1),
        calculate_mae(net_mlp_bce, X, y_raw, c0, c1),
        calculate_mae(net_slp_mse, X, y_raw, c0, c1),
        calculate_mae(net_slp_bce, X, y_raw, c0, c1)
    ]

    plt.figure(figsize=(9, 6))

    plt.bar(names, values)

    plt.xlabel("Config")
    plt.ylabel("MAE")
    plt.title("Scale Recovery Accuracy [-10, 9]")

```

```

plt.grid(axis="y", alpha=0.3)

plt.tight_layout()
plt.show()

def run_experiments(train_func, X, y, y_raw, seeds, epochs, lr, target_error, c0,
c1):
    results = {
        "epochs": [], "errors": [], "accuracies": [],
        "maes": [], "converged": [], "nets": [], "histories": []
    }

    for seed in seeds:
        np.random.seed(seed)

        net, history, epoch, error, mae, converged = train_func(
            [2, 2, 1], X, y,
            epochs=epochs, lr=lr, target_error=target_error
        )

        accuracy = calculate_accuracy(net, X, y_raw, c0, c1)

        results["epochs"].append(epoch)
        results["errors"].append(error)
        results["accuracies"].append(accuracy)
        results["maes"].append(mae)
        results["converged"].append(converged)
        results["nets"].append(net)
        results["histories"].append(history)

        print(
            f"Seed {seed}: epochs={epoch}, error={error:.6f}, "
            f"accuracy={accuracy:.1f}%, MAE={mae:.6f}, converged={converged}"
        )

    return results

if __name__ == "__main__":

    c0, c1 = -10, 9

    X_raw = np.array([
        [c0, c0],
        [c0, c1],
        [c1, c0],
        [c1, c1]
    ])

    y_raw = np.array([

```

```

        [c0],
        [c1],
        [c1],
        [c0]
    ])

X = normalize(X_raw, c0, c1)

y_mse = np.array([
    [0],
    [1],
    [1],
    [0]
])

y_bce = y_mse.copy()

epochs = 200000
lr = 0.01
target_error = 0.01
seeds = [1, 2, 3, 4, 5]

print("\n=====")
print("MLP 2-2-1 + MSE")
print("=====")

mse = run_experiments(
    train_network,
    X, y_mse, y_raw,
    seeds, epochs, lr, target_error, c0, c1
)

print("\n=====")
print("MLP 2-2-1 + BCE")
print("=====")

bce = run_experiments(
    bce_train_network,
    X, y_bce, y_raw,
    seeds, epochs, lr, target_error, c0, c1
)

net_mlp_mse = mse["nets"][0]
history_mlp_mse = mse["histories"][0]

net_mlp_bce = bce["nets"][0]
history_mlp_bce = bce["histories"][0]

print("\n=====")
print("FINAL RESULTS (Seed 1)")
print("=====")

```

```

print(
    f"MSE: epochs={mse['epochs'][0]}, "
    f"error={mse['errors'][0]:.6f}, "
    f"accuracy={mse['accuracies'][0]:.1f}%, "
    f"MAE={mse['maes'][0]:.6f}, "
    f"converged={mse['converged'][0]}"
)

print(
    f"BCE: epochs={bce['epochs'][0]}, "
    f"error={bce['errors'][0]:.6f}, "
    f"accuracy={bce['accuracies'][0]:.1f}%, "
    f"MAE={bce['maes'][0]:.6f}, "
    f"converged={bce['converged'][0]}"
)

print("\n=====")
print("5-RUN STATISTICS")
print("=====")

for name, result in [("MSE", mse), ("BCE", bce)]:
    print(f"\n{name}:")
    print(f"Epochs: {result['epochs']}")
    print(f"Errors: {result['errors']}")
    print(f"Converged: {result['converged']}")
    print(f"Successful runs: {sum(result['converged'])}/5")

plot_convergence(
    history_mlp_mse,
    history_mlp_bce,
    target_error
)

plot_epoch_scatter(
    mse["epochs"],
    mse["converged"],
    bce["epochs"],
    bce["converged"]
)

plot_decision_boundary(
    net_mlp_mse,
    "Decision surface: MLP 2-2-1 + MSE",
    c0, c1
)

plot_decision_boundary(
    net_mlp_bce,
    "Decision surface: MLP 2-2-1 + BCE",

```

```

        c0, c1
    )

plt.figure(figsize=(8, 6))

plt.bar(
    ["MLP + MSE", "MLP + BCE"],
    [mse["maes"][0], bce["maes"][0]]
)

plt.xlabel("Configuration")
plt.ylabel("MAE")
plt.title("Scale recovery accuracy [-10, 9]")
plt.grid(axis="y", alpha=0.3)
plt.tight_layout()
plt.show()

print("\n=====")
print("TEST CASES")
print("=====")

test_cases = [
    (-10, -10),
    (-10, 9),
    (9, -10),
    (9, 9),
    (0, 0),
    (5, -5),
    (-5, 5)
]

for a, b in test_cases:

    print(f"\n--- Point A={a}, B={b} ---")

    print("\nMSE:")
    predict_point(net_mlp_mse, a, b, c0, c1)

    print("\nBCE:")
    predict_point(net_mlp_bce, a, b, c0, c1)

print("\n=====")
print("INTERACTIVE MODE")
print("=====")
print("Enter A and B from [-10; 10].")
print("Enter 'q' to exit.")

while True:

    user_input = input("\nInput A and B: ").strip()

```

```

if user_input.lower() in ["q", "exit", "ВЫХОД"]:
    print("Exit...")
    break

try:
    parts = user_input.split()

    if len(parts) != 2:
        print("Error: enter exactly two numbers!")
        continue

    a, b = map(float, parts)

    if not (-10 <= a <= 10 and -10 <= b <= 10):
        print("Error: A and B must be in [-10; 10]!")
        continue

    print(f"\n--- Point A={a}, B={b} ---")

    print("\nMLP 2-2-1 + MSE:")
    predict_point(net_mlp_mse, a, b, c0, c1)

    print("\nMLP 2-2-1 + BCE:")
    predict_point(net_mlp_bce, a, b, c0, c1)

except ValueError:
    print("Error: invalid input!")

```

4. Таблица результатов:

Конфигурация	Seed	Эпохи	Итоговая ошибка	Accuracy	MAE [c0; c1]	Сходимость
A — MSE	1	88 691	0.01000	100%	1.3378	Да
A — MSE	2	106 738	0.01000	100%	1.3275	Да
A — MSE	3	200 000	0.25755	50%	5.3303	Нет
A — MSE	4	73 537	0.01000	100%	1.3424	Да
A — MSE	5	126 941	0.01000	100%	1.3389	Да
B — BCE	1	109 510	0.01000	100%	0.0474	Да
B — BCE	2	113 425	0.01000	100%	0.0474	Да
B — BCE	3	128 462	0.01000	100%	0.0474	Да
B — BCE	4	105 658	0.01000	100%	0.0474	Да
B — BCE	5	192 151	0.01000	100%	0.0474	Да

5. Блок визуализации: график сходимости:

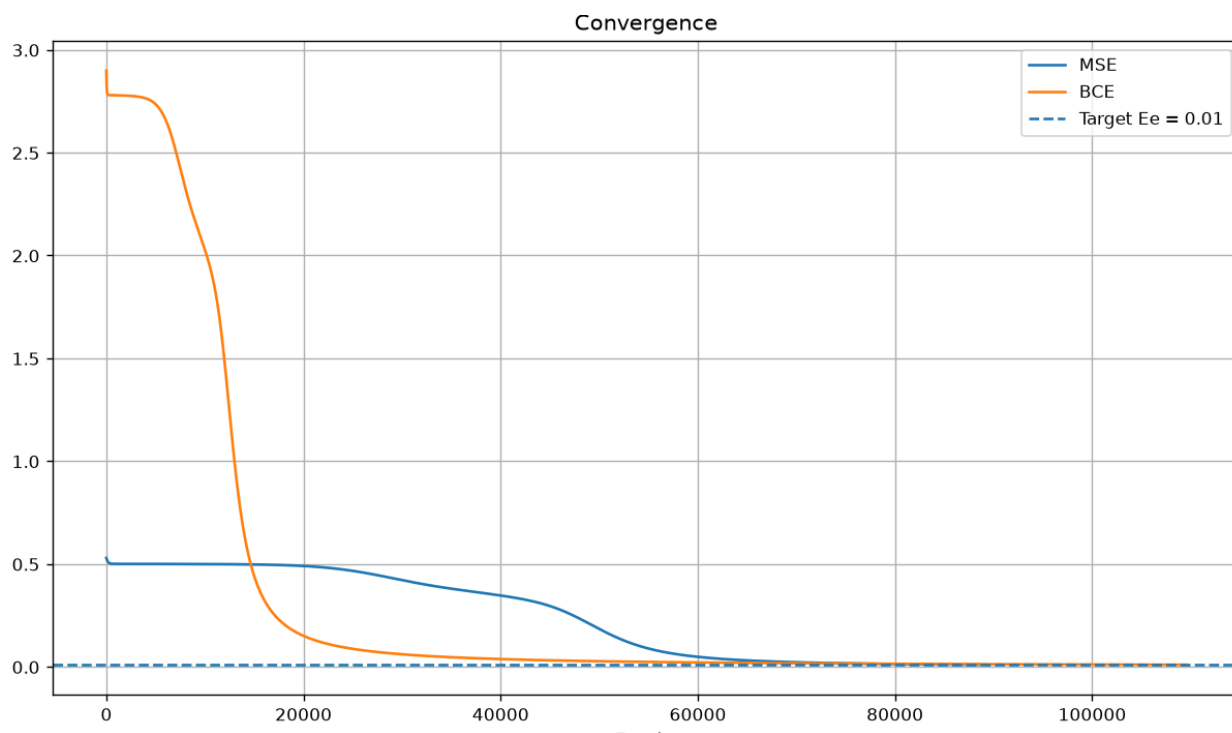
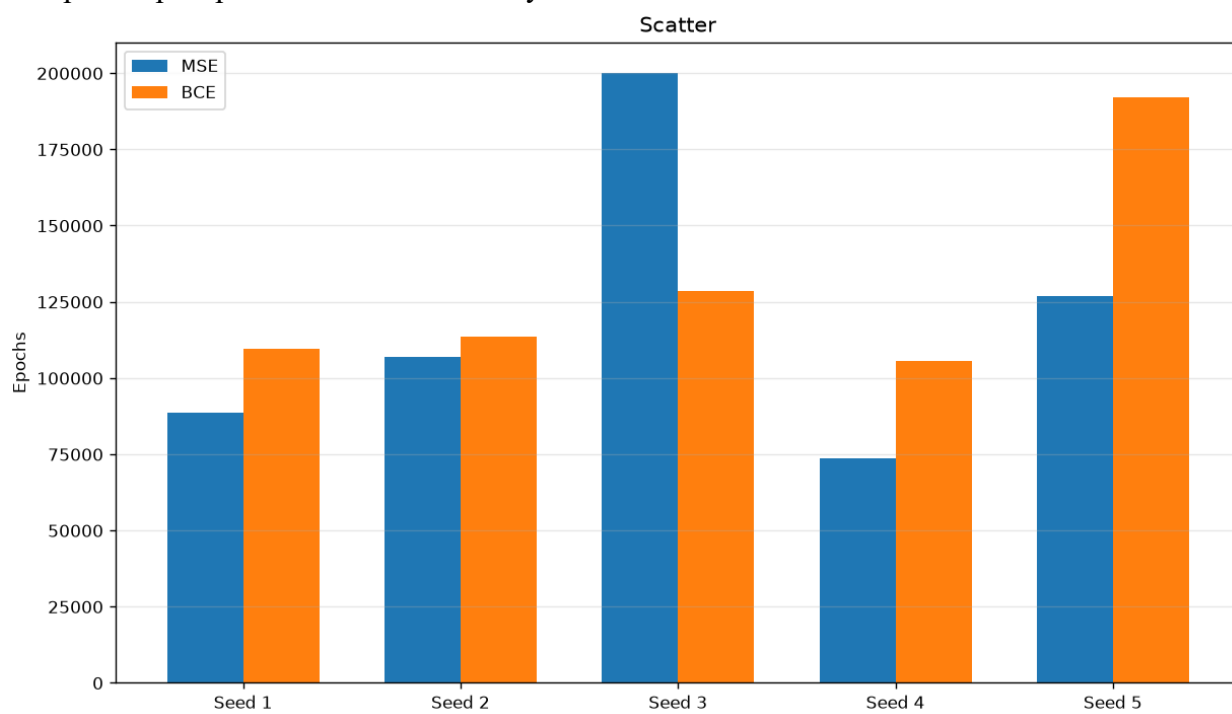
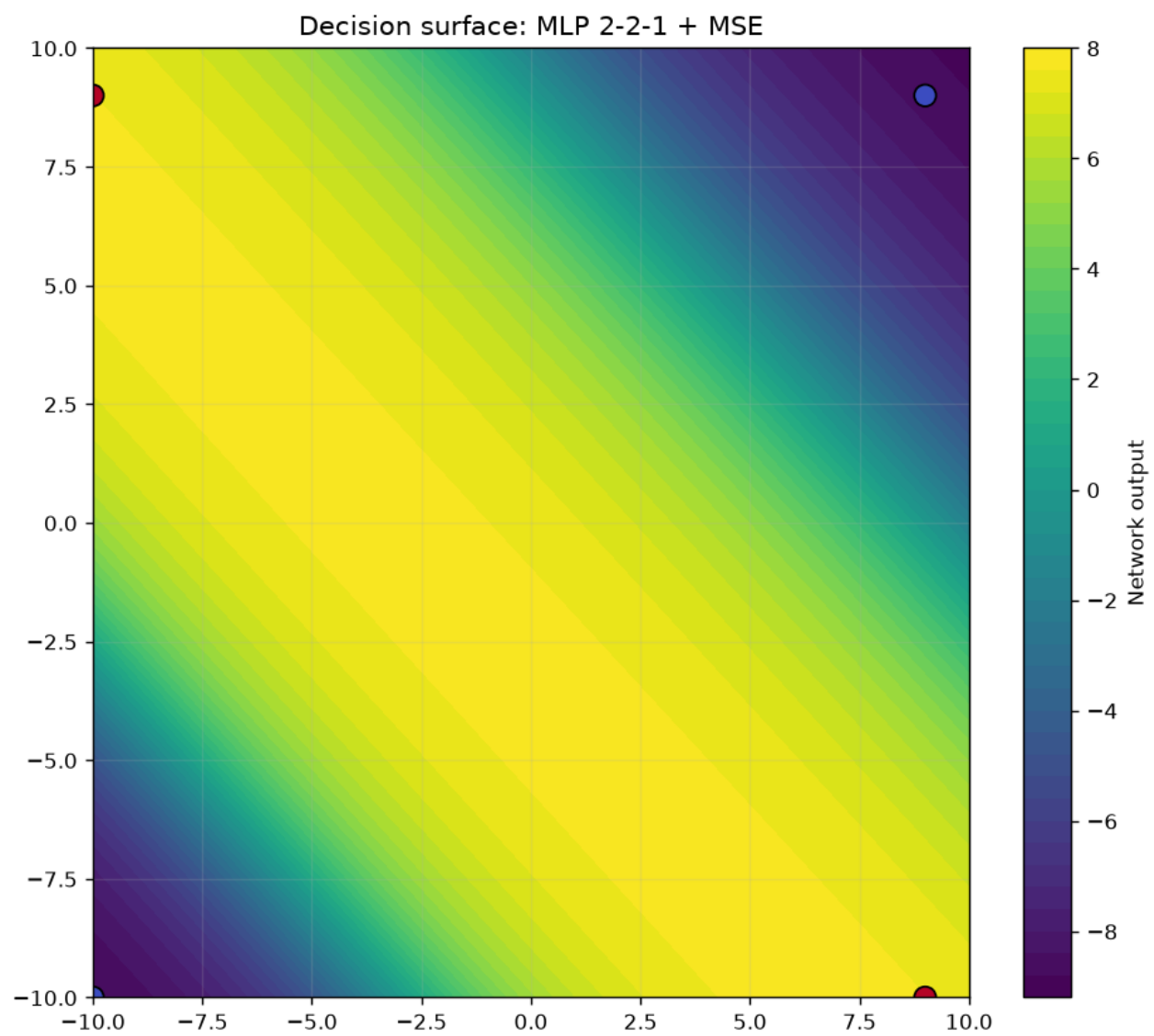


диаграмма разброса числа эпох по запускам



визуализация разделяющей поверхности скрытого слоя для обеих конфигураций



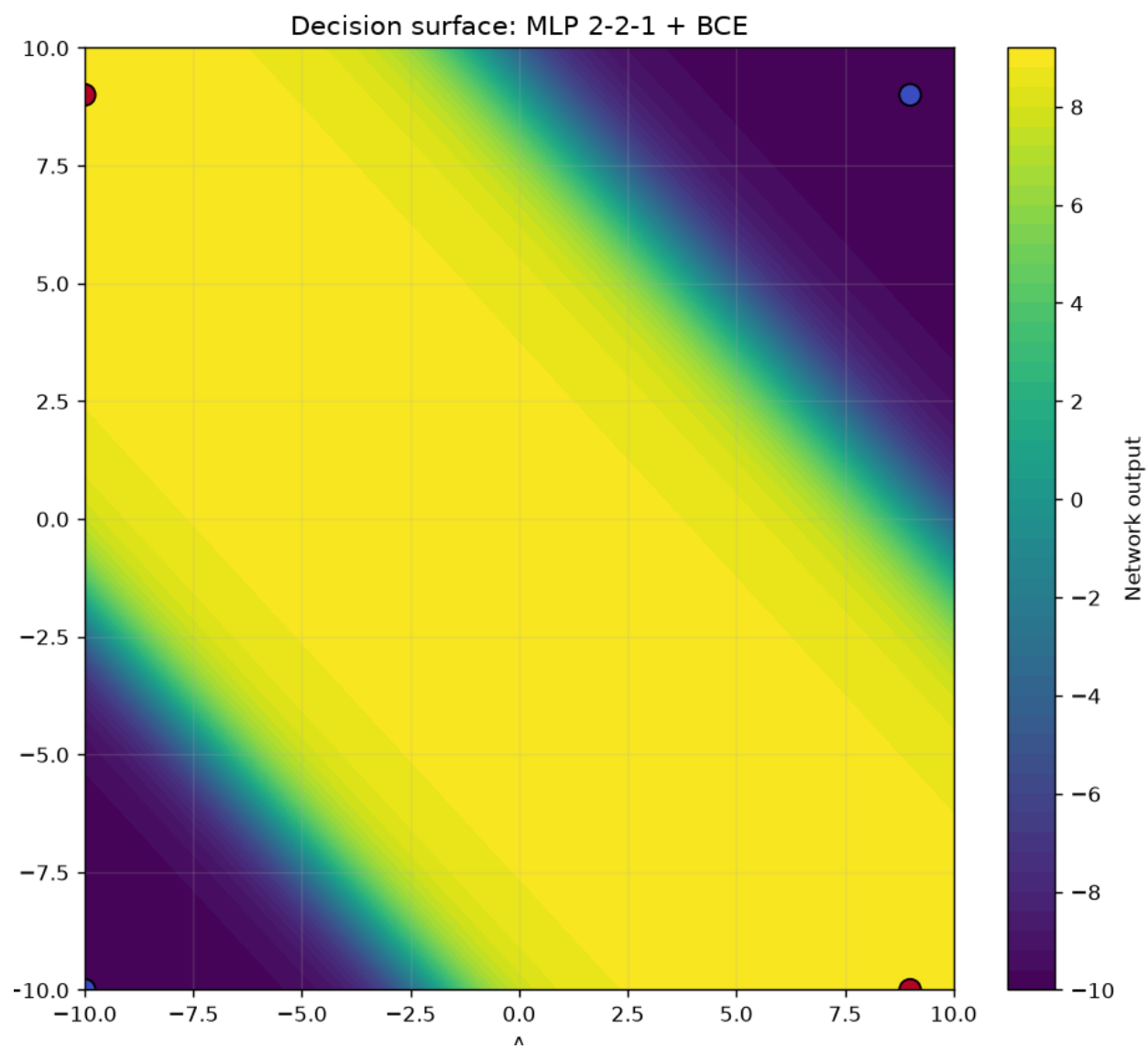
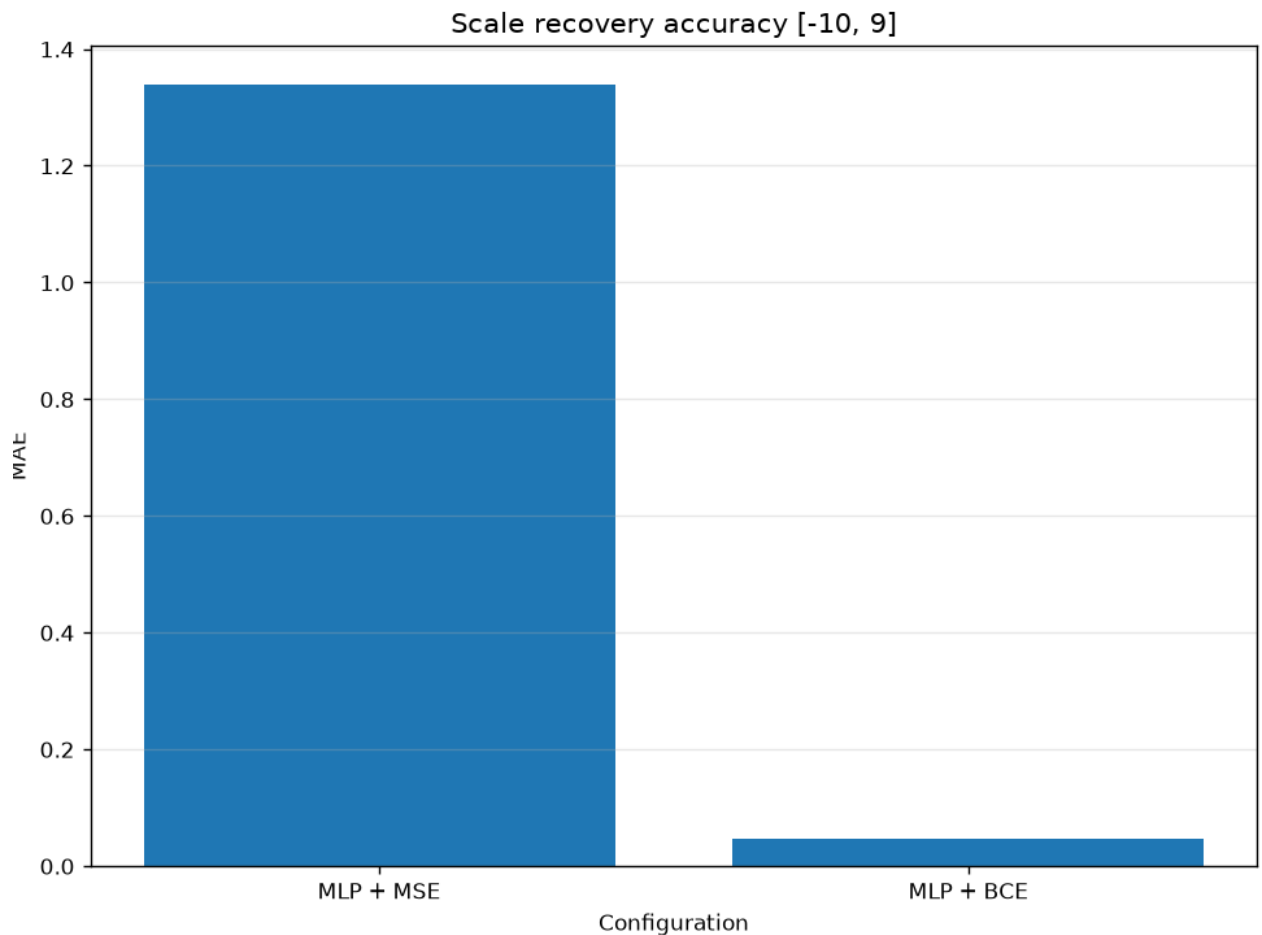


диаграмма сравнения точности восстановления шкалы



6. Результаты работы обученной сети в режиме функционирования (не менее 6–7 проверочных примеров, включая примеры вне обучающей выборки).

Enter A and B from [-10; 10].
Enter 'q' to exit.

Input A and B: 1 1

--- Point A=1.0, B=1.0 ---

MLP 2-2-1 + MSE:
Input: A=1.0, B=1.0
Normalized output: 0.8909
Original scale output: 6.9268
Predicted class: 9

MLP 2-2-1 + BCE:
Input: A=1.0, B=1.0
Normalized output: 0.9949
Original scale output: 8.9037
Predicted class: 9

Input A and B: 2 2

--- Point A=2.0, B=2.0 ---

MLP 2-2-1 + MSE:
Input: A=2.0, B=2.0
Normalized output: 0.8387
Original scale output: 5.9355
Predicted class: 9

MLP 2-2-1 + BCE:
Input: A=2.0, B=2.0
Normalized output: 0.9879
Original scale output: 8.7703
Predicted class: 9

Input A and B: 3 3

--- Point A=3.0, B=3.0 ---

MLP 2-2-1 + MSE:
Input: A=3.0, B=3.0
Normalized output: 0.7476
Original scale output: 4.2045
Predicted class: 9

MLP 2-2-1 + BCE:
Input: A=3.0, B=3.0
Normalized output: 0.9549
Original scale output: 8.1434
Predicted class: 9

Input A and B: 4 4

--- Point A=4.0, B=4.0 ---

MLP 2-2-1 + MSE:
Input: A=4.0, B=4.0
Normalized output: 0.6082
Original scale output: 1.5561
Predicted class: 9

MLP 2-2-1 + BCE:
Input: A=4.0, B=4.0
Normalized output: 0.7762
Original scale output: 4.7475
Predicted class: 9

Input A and B: 5 5

--- Point A=5.0, B=5.0 ---

MLP 2-2-1 + MSE:

Input: A=5.0, B=5.0

Normalized output: 0.4386

Original scale output: -1.6658

Predicted class: -10

MLP 2-2-1 + BCE:

Input: A=5.0, B=5.0

Normalized output: 0.3077

Original scale output: -4.1539

Predicted class: -10

Input A and B: 6 6

--- Point A=6.0, B=6.0 ---

MLP 2-2-1 + MSE:

Input: A=6.0, B=6.0

Normalized output: 0.2844

Original scale output: -4.5972

Predicted class: -10

MLP 2-2-1 + BCE:

Input: A=6.0, B=6.0

Normalized output: 0.0598

Original scale output: -8.8636

Predicted class: -10

7. Вывод с ответами на все поставленные ниже вопросы.

Вывод: На основе многослойного персептрона архитектуры 2-2-1, реализованного в Лабораторной работе №1 для задачи XOR с числовыми классами c_0 и c_1 из диапазона $[-10; 10]$, реализовал альтернативную функцию потерь – бинарную кросс-энтропию (Binary Cross-Entropy, BCE) – и провел сравнительный анализ обучения сети с MSE (Лабораторная работа №1) и с BCE по скорости сходимости, качеству обучения и устойчивости к выбору начальных весов.

Вопросы для вывода

- 1. Какая из функций потерь (MSE или BCE) обеспечила более быструю и/или более устойчивую сходимость на задаче XOR с числовыми классами? Согласуется ли этот результат с известным теоретическим преимуществом BCE для задач классификации?**

В проведённых экспериментах BCE показала более устойчивую сходимость: успешно завершились все 5 запусков, тогда как для MSE — 4 из 5. При этом по скорости однозначного преимущества нет: MSE в успешных запусках достигала порога за 73 537–126 941 эпоху, а BCE — за 105 658–192 151 эпоху. Полученный результат согласуется с теоретическим преимуществом BCE для бинарной классификации: при сигмоидальном выходе её градиент имеет более простой вид $y-t$, что позволяет эффективнее корректировать ошибку классификации.

- 2. Как нормализация целевых значений (приведение c_0/c_1 к 0/1) повлияла на процесс обучения по сравнению с обучением напрямую на числах из диапазона $[-10; 10]$ (как в ЛР №1 с MSE)? Изменилась ли скорость и стабильность сходимости?**

Нормализация целевых значений до диапазона 0/1 сделала обучение более подходящим для сигмоидального выхода сети и улучшила стабильность сходимости. В отличие от обучения на исходных

значениях $[-10;10]$, выход сигмоиды ограничен диапазоном $(0;1)$, поэтому при нормализации ошибка и градиенты становятся более согласованными с диапазоном выхода. В проведённых экспериментах BCE со нормализованными целями сошлась во всех 5 запусках, а MSE — в 4 из 5, однако по скорости сходимости однозначного преимущества нормализация не показала.

- 3. Какова точность восстановления исходной числовой шкалы $[c_0; c_1]$ (средняя абсолютная ошибка) при использовании BCE с последующим пересчётом $\hat{y} \rightarrow y_{\text{real}}$, по сравнению с прямым обучением на MSE, где сеть сразу выдаёт значение в нужной шкале? Есть ли здесь компромисс между удобством классификации (BCE) и точностью регрессии к исходной шкале?**

При использовании BCE средняя абсолютная ошибка после обратного пересчёта в исходную шкалу составила около 0.047, тогда как для MSE — около 1.34. Таким образом, в проведённом эксперименте BCE обеспечила более точное восстановление исходных числовых значений. Поэтому заметного компромисса между классификацией и точностью восстановления в данном случае не наблюдается: BCE одновременно дала 100% точность классификации и меньшую ошибку на исходной шкале.

- 4. Насколько устойчивы обе конфигурации к случайной инициализации весов (по результатам серии из 5 запусков)? Наблюдались ли случаи недостижения критерия остановки или попадания в локальный минимум?**

По результатам 5 запусков BCE показала более устойчивую сходимость: критерий остановки был достигнут во всех 5 из 5 запусков. Для MSE критерий был достигнут в 4 из 5 запусков, а в одном запуске сеть за 200 000 эпох остановилась с ошибкой 0.25755 и точностью 50%. Такой результат может свидетельствовать о попадании обучения в неудачную область параметров (локальный минимум или плато). При этом случайная инициализация заметно повлияла на скорость сходимости обеих конфигураций.

- 5. Подтвердилось ли на данном варианте (со своими c_0 и c_1) сделанное в ЛР №1 наблюдение о принципиальной линейной неразделимости задачи XOR независимо от того, какие конкретно числовые значения используются для кодирования классов 0 и 1?**

Да, наблюдение подтвердилось. Независимо от выбранных числовых значений классов $c_0=-10$ и $c_1=9$, задача XOR остаётся нелинейно разделимой в исходном пространстве признаков. Поэтому изменение

числового кодирования классов не устраняет принципиальную проблему XOR — для её решения требуется многослойная сеть с нелинейными функциями активации.

- 6. Как соотносится выбор функции потерь (MSE vs BCE) с общим выводом из предыдущей лабораторной о необходимости многослойной архитектуры для XOR – влияет ли тип функции потерь на этот вывод, или он определяется исключительно архитектурой сети?**

Тип функции потерь не меняет вывод о необходимости многослойной архитектуры для решения XOR. Это определяется линейной неразделимостью задачи: однослойная сеть не может разделить классы XOR одной границей независимо от выбора MSE или BCE. MSE и BCE влияют на процесс обучения и скорость сходимости, но для корректного решения XOR в данной работе используется многослойная сеть 2-2-1.

- 7. Что показывает визуализация разделяющей поверхности на плоскости (A, B) для обеих конфигураций? Совпадает ли форма границы между классами с ожидаемой для XOR (пересечение двух полуплоскостей, формируемых скрытым слоем), и отличается ли она визуально между конфигурацией А (MSE) и конфигурацией Б (BCE)?**

Визуализация показывает, что обе конфигурации формируют нелинейную границу, разделяющую четыре точки XOR: два противоположных угла относятся к одному классу, а два других — к другому. В целом форма соответствует ожидаемой для XOR структуре, создаваемой комбинацией границ скрытого слоя. Визуально границы MSE и BCE немного отличаются по положению и крутизне, поскольку веса обучаются по разным функциям ошибки, но обе конфигурации реализуют один и тот же принцип разделения классов.